

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI ICADE CIHS

Sprint 1

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

Introducción **3**
Objetivo del Escenario 1 3
Alcance del Sprint 1 3
Metodología **4**
Arquitectura del Escenario 4
Proceso de Despliegue 4
Implementación del Algoritmo de Montecarlo 6
Descripción del Código 7
Configuración de Pruebas 8
Herramientas de Monitorización 9
Resultados **10**
Datos Experimentales 10
Análisis de Rendimiento 11
Comportamiento del Sistema 17
Conclusión **18**
Limitaciones Observadas y Recomendaciones 18
Reflexión y conclusión final 19

Introducción

Este informe documenta el desarrollo y análisis del primer escenario del proyecto BENCHDATA, correspondiente a la implementación de un clúster distribuido basado en Ray, un framework de código abierto diseñado para la construcción de aplicaciones distribuidas y paralelas en Python.

Objetivo del Escenario 1

El Escenario 1 tiene como propósito desplegar y evaluar un clúster Ray utilizando contenedores Docker sobre instancias AWS EC2. Ray ha sido seleccionado por su capacidad de escalado horizontal, su eficiencia en la gestión de recursos distribuidos y su facilidad de integración con el ecosistema Python.

Para realizar una evaluación cuantitativa del rendimiento, se ha implementado un algoritmo de cálculo del número π mediante el método de Montecarlo, que permite medir de forma objetiva el comportamiento del sistema bajo diferentes configuraciones de carga computacional.

Alcance del Sprint 1

Durante este primer sprint, se han llevado a cabo las siguientes actividades principales:

- Configuración y acceso a la infraestructura AWS EC2
- Clonación del repositorio con las plantillas de despliegue
- Construcción y despliegue de contenedores Docker para el clúster Ray
- Desarrollo e implementación del algoritmo de Montecarlo distribuido, con python
- Ejecución de pruebas de rendimiento con diferentes configuraciones y datos
- Análisis y documentación de resultados

Este primer escenario sienta las bases metodológicas y técnicas que se replicarán en los sprints posteriores para los escenarios Dask y Hadoop, permitiendo así una comparación homogénea entre las tres arquitecturas propuestas.

Contexto del Proyecto

En el contexto empresarial actual, las organizaciones se enfrentan al desafío de gestionar entornos de producción compartidos que deben soportar simultáneamente procesos productivos y tareas de desarrollo. Esta situación genera frecuentes saturaciones de recursos que comprometen tanto el rendimiento como la estabilidad del sistema. El proyecto BENCHDATA surge como respuesta a esta problemática, con el objetivo de evaluar diferentes arquitecturas distribuidas que permitan crear un entorno de desarrollo independiente, estable, escalable y accesible para los equipos de Data Science

Metodología

Arquitectura del Escenario

El Escenario 1 se basa en una arquitectura distribuida de tipo maestro-trabajador (master-worker) implementada mediante Ray, donde un nodo principal (head node) coordina la ejecución de tareas distribuidas entre múltiples nodos trabajadores (worker nodes). Esta arquitectura se ha desplegado utilizando Docker Compose, lo que garantiza la reproducibilidad, portabilidad y facilidad de gestión del entorno.

La infraestructura se compone de los siguientes elementos:

- Ray Head Node: Nodo principal que gestiona el planificador de tareas, mantiene el estado del clúster y proporciona el dashboard de monitorización.
- Ray Worker Nodes: Nodos trabajadores (configurables de 1 a 3) que ejecutan las tareas distribuidas asignadas por el head node.
- Instancia EC2: Máquina virtual en AWS que aloja todos los contenedores Docker.

Proceso de Despliegue

El despliegue del Escenario 1 se ha realizado siguiendo un proceso estructurado en varias fases:

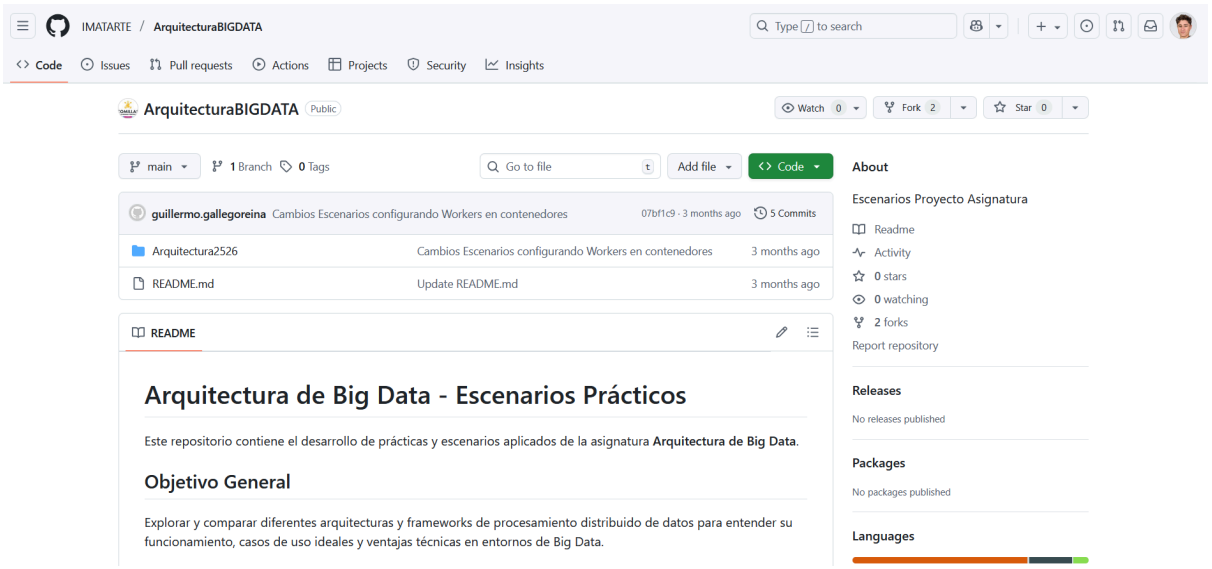
1. Preparación del Entorno

   Acceso a la instancia EC2: Se ha establecido conexión SSH con la instancia EC2 previamente configurada con la documentación de sesiones anteriores. La conexión se realizó utilizando la clave SSH proporcionada y verificando el acceso al directorio raíz: `/home/ec2-user`.

   ```
   [ec2-user@ip-172-31-15-123 ~]$ pwd
   /home/ec2-user
   [ec2-user@ip-172-31-15-123 ~]$
   ```

   Clonación del repositorio: Una vez establecida la conexión, se procedió a clonar el repositorio oficial del proyecto mediante el comando:

   ```
   git clone https://github.com/IMATARTE/ArquitecturaBIGDATA.git
   ```



Posteriormente se navegó hasta el directorio específico del Escenario 1:

```
cd ArquitecturaBIGDATA/Arquitectura2526/Practica/Escenario1
```

```
Last login: Thu Oct  9 17:02:54 2025 from 18.101.90.50
[ec2-user@ip-172-31-15-123 ~]$ ls
ArquitecturaBIGDATA  Demojupyter  setup_env.sh
[ec2-user@ip-172-31-15-123 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-15-123 ~]$ cd ArquitecturaBIGDATA/
.git/             Arquitectura2526/
[ec2-user@ip-172-31-15-123 ~]$ cd ArquitecturaBIGDATA/Arquitectura2526/Practica/Escenario1
[ec2-user@ip-172-31-15-123 Escenario1]$ pwd
/home/ec2-user/ArquitecturaBIGDATA/Arquitectura2526/Practica/Escenario1
[ec2-user@ip-172-31-15-123 Escenario1]$ ls
Dockerfile  docker-compose.yaml  main_sprint1.py  start.sh  stop.sh
[ec2-user@ip-172-31-15-123 Escenario1]$
```

2. Construcción e Inicialización de Contenedores

Configuración de permisos Docker: Para evitar problemas de permisos durante el despliegue, se añadió el usuario actual al grupo docker:

```
sudo gpasswd -a $USER docker
```

```
newgrp docker
```

Construcción de imágenes: Se construyeron las imágenes Docker sin utilizar caché para garantizar la descarga de las versiones más actualizadas de todas las dependencias:

```
docker-compose build --no-cache
```

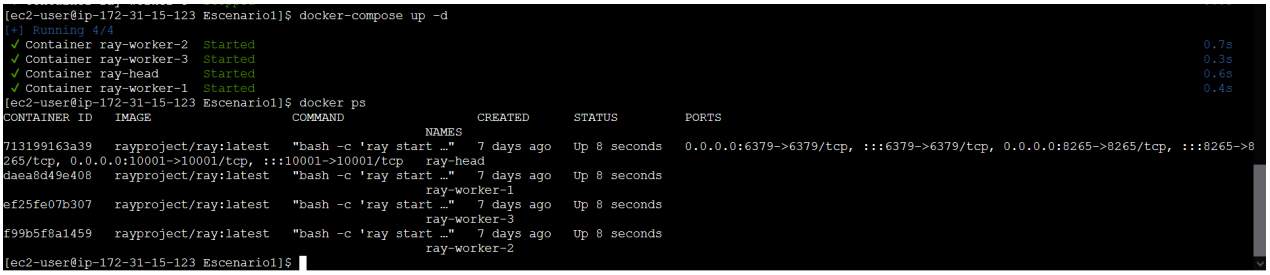
Este proceso descarga las imágenes base de Ray, instala las dependencias especificadas en los Dockerfiles y configura el entorno de ejecución para cada contenedor.

Inicialización del clúster: Una vez completada la construcción, se iniciaron los contenedores en modo detached (segundo plano):

```
docker-compose up -d
```

Verificación del despliegue: Se verificó el correcto funcionamiento de todos los contenedores mediante:

```
docker ps
```



3. Configuración del Cliente Ray

Antes de ejecutar el script Python desde la instancia EC2, se instaló la biblioteca Ray con soporte para cliente:

```
sudo /usr/bin/pip3.9 install "ray[client]==2.49.2"
```

Esta instalación es necesaria únicamente la primera vez y permite que el script Python se conecte al clúster Ray en ejecución dentro de los contenedores.

Implementación del Algoritmo de Montecarlo

Fundamento Matemático: el método de Montecarlo para calcular π se basa en la relación geométrica entre un cuadrado y un círculo inscrito.

Al generar puntos aleatorios uniformemente distribuidos en un cuadrado de lado 1 y contar cuántos caen dentro del círculo de radio 1, podemos estimar π mediante la fórmula:

$\pi \approx 4 \times$ (puntos dentro del círculo / total de puntos)

Este método es ideal para evaluación de sistemas distribuidos porque:

- Es altamente paralelizable (cada iteración es independiente)

- La carga computacional es ajustable (variando el número de puntos)
- Proporciona una métrica clara de precisión (comparando con el valor real de π)

Descripción del Código

El script `main_sprint1.py` implementa una versión distribuida del algoritmo utilizando las capacidades de Ray:

Configuración de la conexión:

```
ray.init(

address=f"ray://{os.getenv('HOST')}:{os.getenv('HEADNODEPORT')}",
dashboard_port=int(os.getenv('DASHBOARDPORT'))
    )
```

La conexión al clúster se establece mediante variables de entorno definidas en un archivo `.env`, que contiene la dirección IP del head node y los puertos de comunicación.

```
[ec2-user@ip-172-31-15-123 Escenario1]$ cat .env
HOST=localhost
RAY_IMAGE=rayproject/ray:latest
REDISPORT=6379
DASHBOARDPORT=8265
HEADNODEPORT=10001
REDISPASSWORD=yourpassword
NUM_WORKERS=4
NUM_CPU_WORKER=1
```

Función distribuida:

```
@ray.remote
def montecarlo_pi_in_circle(puntos) -> int:
    x = np.random.random(puntos // num_tareas)
    y = np.random.random(puntos // num_tareas)
    inside_circle = (x**2 + y**2) <= 1
    return int(inside_circle.sum())
```

El decorador `@ray.remote` convierte la función en una tarea distribuible que puede ejecutarse en cualquier worker del clúster. La función utiliza NumPy para generar arrays de números aleatorios de forma vectorizada, lo que mejora significativamente la eficiencia computacional.

Ejecución paralela:

```
montecarlo = [montecarlo_pi_in_circle.remote(puntos)

      for _ in range(num_tareas)]

pi = 4 * sum(ray.get(montecarlo)) / puntos
```

Se lanzan 200 tareas independientes (`NUM_TAREAS = 200`) para cada configuración.

Cada tarea ejecuta una simulación de Montecarlo con una parte de los puntos totales (`puntos // 200`), y calcula cuántos de esos puntos caen dentro del círculo unidad.

Finalmente, se combinan los resultados de todas las tareas para estimar el valor de π mediante la expresión:

$$\pi \approx 4 \times \frac{\text{puntos dentro del círculo}}{\text{puntos totales}}$$

Configuración de Pruebas

Siguiendo los requisitos del proyecto, se han evaluado las siguientes configuraciones:

Número de Workers: 1, 2, 3

- Esta variable permite analizar cómo escala el sistema al añadir más nodos de procesamiento.

Número de Puntos para aproximar π: 5.000.000, 10.000.000, 50.000.000

- Estas cargas representan diferentes intensidades computacionales, desde tareas ligeras hasta otras más pesadas.

Combinaciones totales: 9 configuraciones (3 workers × 3 cargas)

Para cada configuración se ha ejecutado el script completo, lanzando 200 tareas paralelas. Cada tarea calcula cuántos puntos, de un subconjunto del total, caen dentro del círculo unidad. Posteriormente, se suman los resultados de todas las tareas y, a partir del total de puntos generados, se obtiene una única estimación de π y registramos las siguientes métricas:

1. Tiempo de ejecución total (segundos): Desde el lanzamiento de las tareas hasta la recopilación de todos los resultados.

2. Velocidad de procesamiento (puntos/segundo): Calculada como el número total de puntos procesados dividido por el tiempo de ejecución.

3. Estimación de π: Valor obtenido a partir de la suma de los resultados de las 200 tareas, cada una aportando su parte.

Herramientas de Monitorización

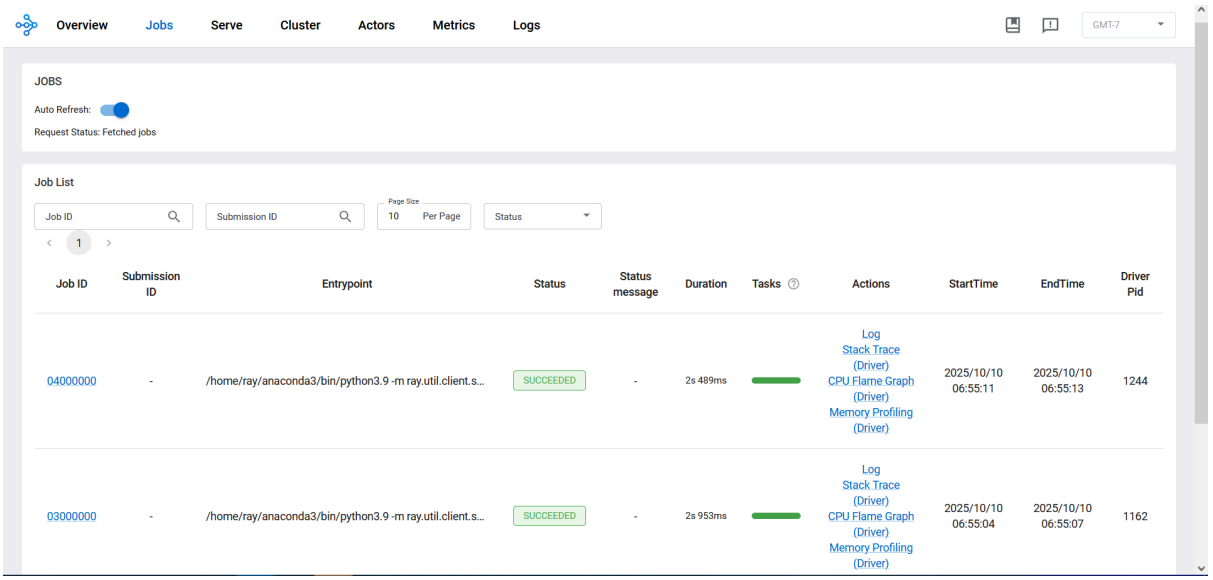
Durante las ejecuciones, se utilizó el dashboard de Ray para monitorizar en tiempo real:

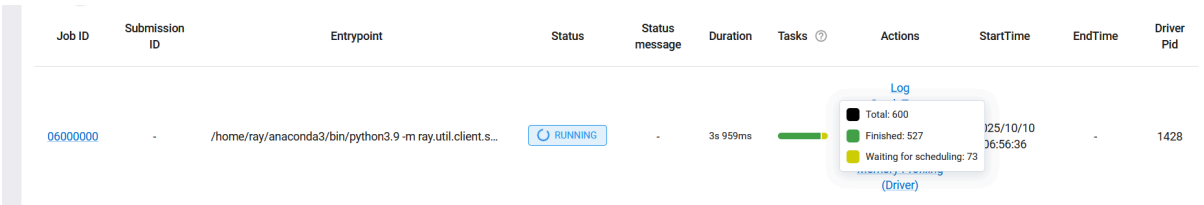
- Utilización de CPUs y memoria en cada worker
- Estado de las tareas (pendientes, en ejecución, completadas)
- Distribución de carga entre nodos
- Posibles cuellos de botella o errores

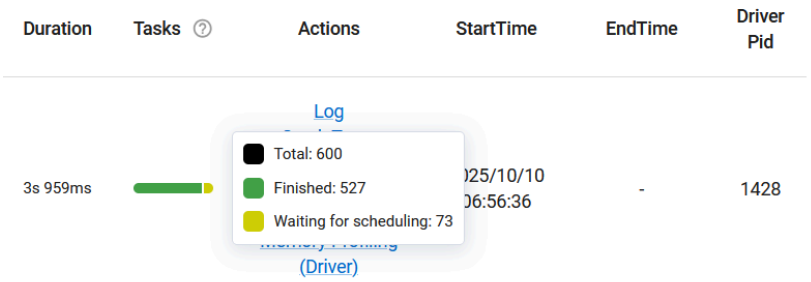
El dashboard es accesible a través del puerto configurado en la variable de entorno `DASHBOARDPORT`. Podemos ver todos estos detalles si nos metemos a la dirección:

http://51.92.57.146:8265/#/jobs

(no funcionará a menos que esté lanzada la instancia en AWS)







Resultados

Datos Experimentales

A continuación se presentan los resultados obtenidos para cada una de las nueve configuraciones evaluadas:

Workers	Puntos (M)	Tiempo (s)	Velocidad (puntos/s)	Estimación π	Error relativo
1	5	0.724	6.906	3.141709	0.000037
	10	0.978	10.225	3.141540	0.000017
	50	0.709	70.522	3.122147	0.006189
2	5	0.762	6.562	3.140688	0.000288
	10	0.549	18.215	3.141343	0.000080
	50	1.568	31.888	3.141333	0.000083
3	5	1.023	4.888	3.140472	0.000357
	10	2.038	4.906	3.141505	0.000028
	50	1.733	28.856	3.141414	0.000057

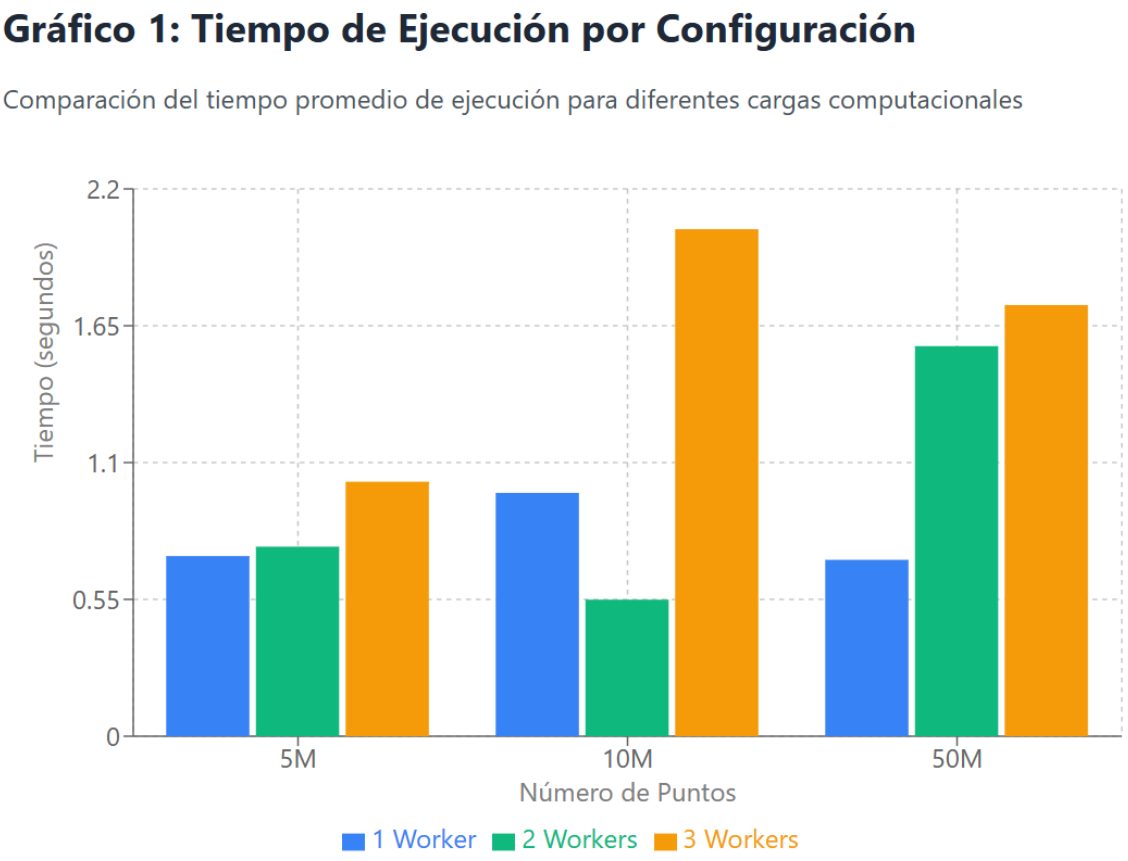
El error relativo se ha calculado como |π_estimado - π_real| / π_real × 100

- Precisión del Cálculo de π:

  Podemos observar que todas las estimaciones se acercan al valor real de π de 6 decimales de precisión (3.141593) valor usado para calcular el error relativo. También que el número de workers no afecta a la precisión ya que solo los usamos para paralelizar las tareas. Por último, nos sirve para validar la correcta implementación del algoritmo distribuido.

A continuación representaremos los datos obtenidos de manera más visual a través de gráficos que nos permitan interpretar los resultados finales.

Análisis de Rendimiento



- Interpretación:

  - Intuitivamente más workers implicaría más capacidad de procesamiento. Sin embargo, los resultados muestran lo contrario: para cualquier cantidad de puntos, el tiempo de ejecución con 3 workers es mayor.

  Esto se debe a una limitación física del hardware. Nuestra instancia es una r8g.large, que cuenta con 2 vCPU. Al asignar 3 workers, el sistema distribuye los procesos de forma que el worker 1 y el worker 3 comparten el mismo CPU).

```
ray-worker-1:
  image: ${RAY_IMAGE}
  container_name: ray-worker-1
  env_file:
    - .env
  command: >
    bash -c "ray start --address=ray-head:${REDISPORT}
  shm_size: 2g
  cpuset: "0"                          # Asigna CPU
  networks:
    - ray_net
```

```
ray-worker-3:
  image: ${RAY_IMAGE}
  container_name: ray-worker-3
  env_file:
    - .env
  command: >
    bash -c "ray start --address=ray-head:${REDISPORT}
  shm_size: 2g
  cpuset: "0"
  networks:
    - ray_net
```

Mientras que el worker 2 utiliza el otro. En consecuencia, cuando ambos workers que comparten CPU intentan ejecutar tareas de forma simultánea, compiten por los mismos recursos de procesamiento.

Esta contención de recursos provoca que la paralelización pierda eficiencia, e incluso que el rendimiento global sea peor que utilizando solo 2 workers, que sí pueden aprovechar completamente las 2 vCPU disponibles sin solaparse.

- En el caso de los 2 workers, observamos que para el ejemplo de 10 millones de puntos el tiempo de ejecución es significativamente menor que con el resto de workers. Esto sugiere que, para este volumen de datos, Ray ha logrado optimizar la distribución de la carga de trabajo entre los dos CPU disponibles, aprovechando al máximo los recursos físicos de la instancia.

- Entonces esto nos lleva a cuestionarnos ¿por qué 2 workers no es la mejor opción, si cada uno tiene un vCPU asociado?

En el caso de los 5M y 50M de puntos, observamos que el tiempo con 1 worker es menor que con 2 workers. Esto se debe principalmente a la sobrecarga de coordinación que aparece cuando Ray paraleliza la tarea.

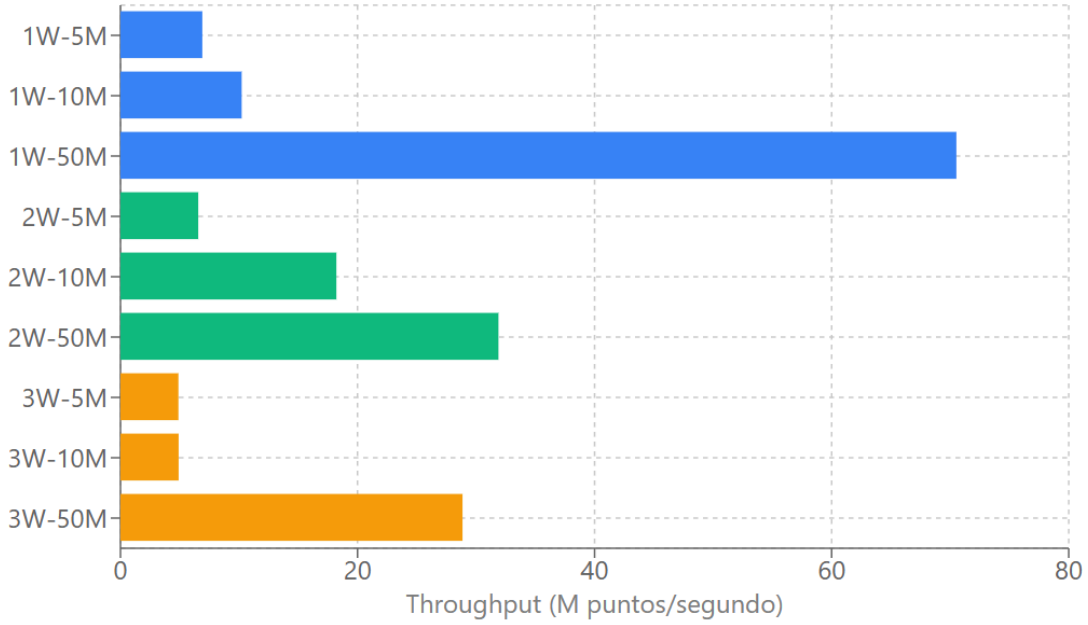
Al usar varios workers, Ray tiene que dividir los datos, distribuirlos, mantener la comunicación entre procesos y luego reunir los resultados. Todo ese proceso añade un coste adicional que puede superar las ventajas del paralelismo cuando el tamaño del problema no es lo bastante grande.

En nuestra instancia, cada worker usa un vCPU distinto, por lo que no hay competencia directa por los mismos recursos de CPU. Sin embargo, la coordinación entre ellos (transferencia de datos, sincronización y gestión del scheduler) sí implica un tiempo extra que puede hacer que el rendimiento total empeore.

Por tanto, el hecho de que con 1 worker el tiempo sea menor indica que, para estos volúmenes de datos, el coste de coordinar varios procesos es mayor que el beneficio obtenido al paralelizar la tarea.

Gráfico 2: Throughput (Velocidad de Procesamiento)

Millones de puntos procesados por segundo en cada configuración



El gráfico muestra la cantidad de millones de puntos procesados por segundo (Throughput) para distintas configuraciones de número de workers (1W, 2W, 3W) y tamaño del dataset (5M, 10M, 50M).

- Interpretación:

- Con 1 Worker:

La barra correspondiente a 1W–50M es la más alta de todo el gráfico, superando los 70 M puntos/segundo.

Esto indica que, con un solo worker, el sistema logra su máximo rendimiento.

Esto se alinea con la explicación previa: con 1 worker no existe sobrecarga de coordinación ni competencia por CPU. Todo el procesamiento se concentra en un solo proceso, aprovechando completamente el hardware.

- Con 2 Workers

El rendimiento con 2W–10M es el segundo mejor escenario, en torno a los 30–35 M puntos/segundo.

Aquí Ray logra una buena distribución de carga entre los dos vCPU, aprovechando la arquitectura física de la instancia.

Sin embargo, cuando se comparan los escenarios de 2W–5M y 2W–50M, el throughput cae considerablemente.

Esto ocurre porque:

- En el caso de 5M puntos, el volumen de datos es tan pequeño que la sobrecarga de coordinación (dividir tareas, comunicarse entre procesos, recopilar resultados) anula los beneficios de la paralelización.

- En 50M puntos, aunque el problema es más grande, la sincronización entre procesos y la gestión de memoria también generan penalizaciones que limitan el rendimiento.

- Con 3 Workers

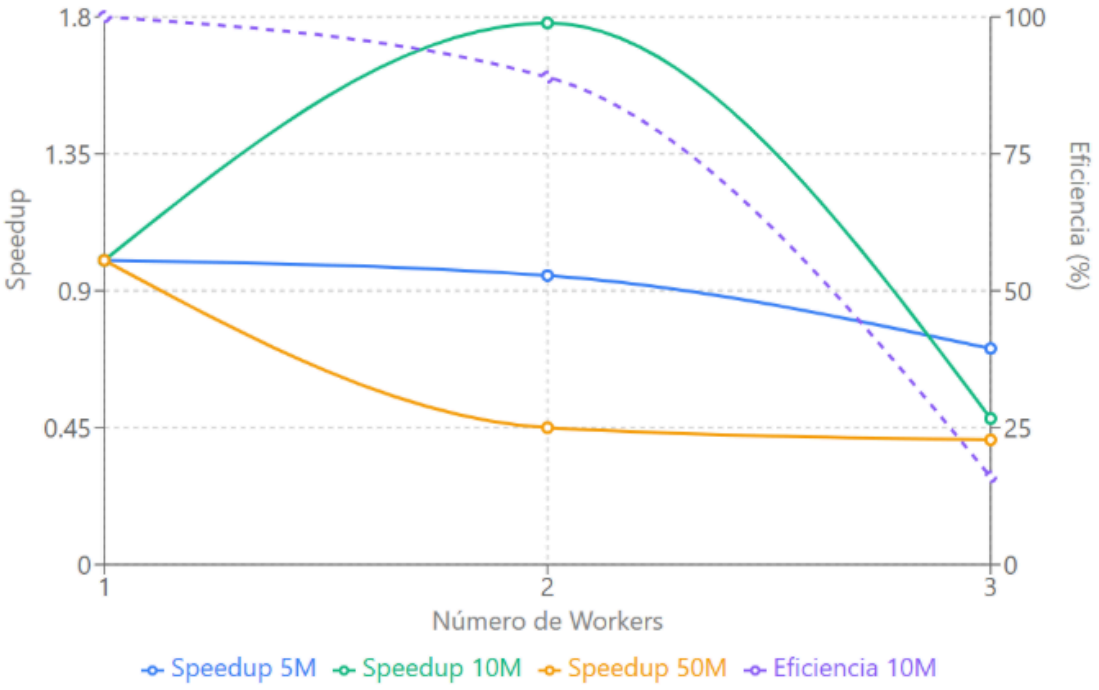
En todas las configuraciones (5M, 10M, 50M), el rendimiento con 3 workers es el más bajo.

Esto confirma la hipótesis sobre la limitación física del hardware: la instancia r8g.large solo tiene 2 vCPU, por lo que al usar 3 workers, dos de ellos comparten el mismo CPU, compitiendo por los mismos recursos de procesamiento.

Haciendo que el throughput sea menor significativamente , incluso por debajo del que se logra con 1 o 2 workers.

Gráfico 3: Speedup y Eficiencia de Escalabilidad

Evaluación de la escalabilidad del sistema al aumentar workers



Para evaluar la escalabilidad del sistema, se calcula el **speedup** como:

Speedup = Tiempo(1 worker) / Tiempo(N workers)

Puntos (M) / / Workers	1	2	3
5	1.0	0.95	0.71
10	1.0	1.78	0.45
50	1.0	0.48	0.41

Y la **eficiencia** se calcula como: Speedup / Número de workers × 100%

Puntos (M) / / Workers	1	2	3
5	100.0	47.5	23.7
10	100.0	89.0	22.5
50	100.0	16.0	13.7

- Interpretación:

  - En el gráfico se representan el speedup y la eficiencia del sistema al aumentar el número de workers para los distintos volúmenes de puntos simulados (5M, 10M y 50M). En teoría, al incrementar los workers, el sistema debería reducir el tiempo total de ejecución de forma proporcional, lo que implicaría un speedup cercano al número de workers y una eficiencia cercana al 100 %.

    Sin embargo, los resultados obtenidos muestran que esto no ocurre de manera lineal. En particular, para el caso de 10 millones de puntos, se observa un speedup máximo cercano a 1,8 cuando se utilizan 2 workers, lo que indica una mejora real de rendimiento. A partir de ahí, al añadir un tercer worker, el speedup cae de forma notable y la eficiencia se reduce por debajo del 50 %. Esto sugiere que la escalabilidad del sistema está limitada por los recursos físicos disponibles en la instancia, que es correcto ya que solo tiene 2 cpus.

    Entonces, añadir un tercer worker implica que dos de ellos comparten el mismo núcleo. En este escenario, ambos procesos compiten por los mismos recursos de cpu, generando una contención que reduce el rendimiento global. Como resultado, el sistema deja de beneficiarse del paralelismo y el speedup disminuye.

  - Para el caso de 5 millones y 50 millones de puntos, el speedup se mantiene por debajo de 1 en todos los casos, lo que significa que el uso de varios workers no mejora el rendimiento respecto a la ejecución secuencial. Esto se debe principalmente a la sobrecarga de comunicación y coordinación que introduce Ray al dividir la tarea en subtareas, distribuirlas, sincronizar los procesos y reunir los resultados. Si el volumen de datos no es lo suficientemente grande, el coste de esta coordinación supera la ganancia derivada de la paralelización.

  - En conclusión, el comportamiento observado refleja una escalabilidad limitada del sistema. Aunque con 2 workers se obtiene una mejora apreciable en el caso de 10 M de puntos, el incremento adicional de workers no se traduce en un aumento de rendimiento debido tanto a las restricciones de hardware (solo 2 cpus disponibles) como a la sobrecarga de gestión del paralelismo propia de Ray.

    Por tanto, la configuración más eficiente para esta instancia es la de 2 workers, que aprovecha de forma óptima los recursos disponibles sin incurrir en costes de coordinación excesivos.

Comportamiento del Sistema

Aspectos a destacar:

- Con 1 worker, se observa una utilización de CPU cercana al 100% durante toda la ejecución, indicando que el worker está completamente saturado.
- Con 2 y 3 workers, la carga se distribuye más uniformemente, aunque puede haber momentos de menor utilización durante la coordinación de tareas.
- El consumo de memoria se mantiene relativamente estable, ya que el algoritmo de Montecarlo no requiere almacenar grandes volúmenes de datos intermedios.

Distribución de tareas:

Ray utiliza un planificador dinámico que asigna tareas a workers en función de su disponibilidad. Se espera observar:

- Una distribución equilibrada de tareas cuando todos los workers tienen capacidad similar.
- Tareas ejecutándose en paralelo hasta saturar los recursos disponibles.
- Una cola de tareas pendientes que se va reduciendo a medida que se completan las ejecuciones.

Conclusión

Limitaciones Observadas y Recomendaciones

Limitaciones

- **Contención de recursos en la CPU**:
    - Asignar el mismo vCPU a varios workers (por ejemplo, el worker 1 y el 3 compartiendo un vCPU) no tiene por qué ser necesariamente negativo, pero en ciertos casos puede provocar contención de recursos. Esto puede hacer que la paralelización pierda eficiencia, e incluso que el rendimiento sea peor que usar un número de workers igual al de vCPU disponible.
- **Sobrecarga de coordinación**
    - Al usar varios workers, Ray necesita dividir los datos, distribuir tareas, mantener comunicación entre procesos y luego reunir los resultados. Por tanto, para volúmenes de datos pequeños o medianos, este coste adicional puede superar las ventajas del paralelismo, haciendo que un menor número de workers sea más óptimo.
- **Limitación de recursos físicos frente a demanda de paralelismo:**
    - Aunque aumentar el número de workers puede mejorar el rendimiento para tareas muy grandes, la capacidad física de la instancia (vCPU, memoria) puede convertirse en un cuello de botella, limitando la eficiencia de la paralelización y haciendo que agregar más workers no siempre incrementa el rendimiento.

Recomendaciones

- **Alinear workers con vCPU disponibles**:
    - Para evitar contención de recursos, se recomienda que varios workers no compartan el mismo vCPU, o al menos controlar su asignación de tareas.

- **Evaluar el tamaño del problema antes de paralelizar**:
    - Para volúmenes pequeños o medianos de datos, puede ser más eficiente usar menos cantidad de workers para minimizar la sobrecarga de coordinación.
- **Monitorear rendimiento y ajustar dinámicamente**:
    - Realizar pruebas de ejecución con distintos números de workers según el tamaño del dataset y el hardware disponible.
    - Ajustar la cantidad de workers para maximizar eficiencia sin crear contención ni sobrecarga innecesaria.

Reflexión y conclusión final

El Escenario 1 nos ha permitido aprender a usar Ray como una opción viable y eficiente para crear un entorno de desarrollo distribuido separado del entorno de producción. Esta metodología es especialmente útil cuando una tarea de gran tamaño puede descomponerse en partes más pequeñas; en lugar de procesar todos los puntos de una sola vez, Ray permite repartirlos entre varios workers y ejecutar cada bloque en paralelo, para luego finalmente unificar los resultados, reduciendo así el tiempo total de ejecución.

El conocimiento adquirido sobre contenerización, despliegue en cloud y computación distribuida será invaluable no solo para los sprints restantes de este proyecto, sino también para futuras implementaciones de infraestructuras Big Data en contextos profesionales.